

Reward, Reset, and Evaluation Design for Sim-to-Real Reinforcement Learning: A Case Study in Quadrotor Gate Racing

Zhuohao Ni* Yuqian (Bruce) Zhang*

University of Pennsylvania

August 25, 2026

Abstract

We report a two-phase reinforcement learning project in which one quadrotor racing task—a seven-waypoint course containing a self-intersecting double-gate structure—was first solved as a pure simulation problem and then re-solved as a transfer problem targeting a physical vehicle tracked by motion capture. The phases share a track, a simulator and a learning algorithm, and almost nothing else: the reward, the observation space, the reset distribution, the randomization strategy and the checkpoint-selection rule were each re-derived, and in four of five cases inverted, once the simulator stopped being the objective and became a hypothesis. Dense shaping with hand-placed virtual targets reached 97.3% three-lap success at 15.68s mean over 5000 randomized environments; a sparse gate-centric reward with an observation space constrained by the deployment interface reached 98.0% at 18.34s under a re-identified dynamics nominal and completed the physical three-lap evaluation in 19.004s, continuing to five correctly ordered laps. Three design objects usually treated as implementation details carried most of the transfer risk for us: the reward predicate’s relationship to the task’s progress predicate, the reset distribution as an explicit choice of where the value function is fit, and the evaluation harness—which failed in both directions, once by being more diverse than deployment and once by being calibrated so closely to measured hardware that it ranked first a policy that managed a single lap in the arena.

1 Introduction

Reinforcement learning for agile flight is usually presented as a transfer problem: train in simulation, randomize the physics, deploy [4, 3, 12]. Framed that way the interesting variable is the randomization distribution and the reported result is a success rate. The framing hides the fact that a policy trained against a simulator whose fidelity is not in question and a policy trained against a simulator standing in for hardware solve different optimization problems, even when the MDP is written identically.

We solved both. The task is fixed: three laps of a seven-waypoint gate course, in order, without crashing. In the *simulation phase* correctness was defined by the simulator, so the rational strategy was to shape as aggressively

as training stability allowed. In the *sim-to-real phase* that simulator became a model with unknown parameters, an observation pipeline with a biased physical counterpart, and evaluation numbers that had to predict an outcome measurable only a handful of times. Three decisions inverted:

1. **Dense $\rightarrow$ sparse reward.** Shaping that specifies *how* to fly encodes a trajectory manifold optimal only under the training dynamics.
2. **Exploit tolerance $\rightarrow$ exploit detection.** In simulation we removed a termination rule punishing reverse gate passes, because tolerating the mistake trained more smoothly; in transfer we added three detectors.
3. **Randomization as insurance $\rightarrow$ randomization as identified uncertainty.** Widening the distribution around an unidentified nominal buys robustness in directions that do not matter.

The fourth change was not an inversion but a constraint: the observation space stopped being a free design variable and became an interface contract with the deployment controller.

Our contributions are empirical rather than algorithmic: we use standard PPO [11] with GAE [10] throughout, and the learning algorithm is not where the difficulty lay. We offer a measured account of (i) a reward-hacking taxonomy induced by a self-intersecting task graph and a general fix, (ii) the reset distribution as a first-class design object including its interaction with actuator state, (iii) a sensitivity analysis that reallocated our randomization budget from dynamics to observations, and (iv) two opposite failure modes of evaluation harness design.

2 Problem Setup

Task and control stack. The vehicle is a Crazyflie-class quadrotor simulated in Isaac Lab [5]. Rigid-body physics runs at 500 Hz; the policy acts at 50 Hz (decimation 10) and emits $a_t = [\bar{T}, \omega_x^{\text{des}}, \omega_y^{\text{des}}, \omega_z^{\text{des}}] \in [-1, 1]^4$, a normalized collective thrust and three body-rate commands, which an inner 500 Hz PID rate controller converts to a desired wrench tracked by a first-order motor model. Episodes last 30s or terminate on crash, capped at three laps. This action interface is the one exposed by the physical stack, and it fixes what *can* be randomized: everything

*These authors contributed equally to this work.

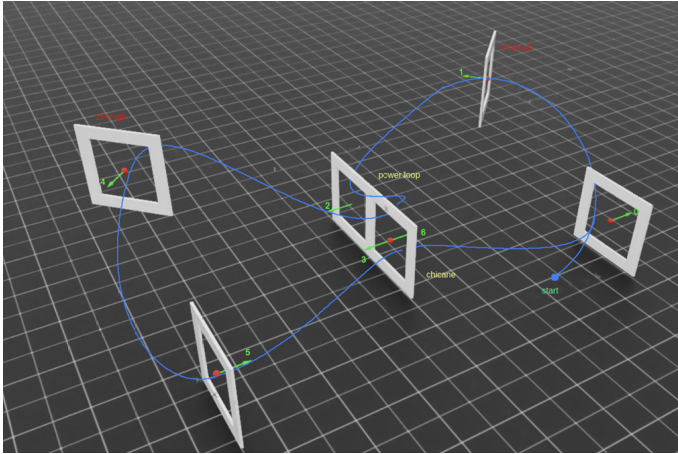


Figure 1: Powerloop race track layout

between action and wrench.

Track geometry. Six waypoints are ordinary; the interesting structure is central. Waypoints 2 and 3 are gate planes 1.25 m apart at $z = 0.75$ m with identical heading, and waypoint 6 is *the same plane as waypoint 3, entered from the opposite side*. The course self-intersects: the state (position, velocity) does not determine which waypoint the agent is pursuing, and one physical gate crossing can be a legitimate pass, an illegitimate reverse pass, or a pass toward a different waypoint index, depending on history. The remaining waypoints sit at $(\pm 1.5, \pm 3.5, 2.0)$ and $(2.0, \pm 3.5, 0.75)$, so each lap couples two wide high-altitude turns to a tight low-altitude transit of the central structure.

Nearly every difficulty in this project traces to that fact. It makes gate-pass detection a stateful predicate rather than a geometric test; it creates a family of reward exploits (Section 5.3); and because the drone must gain and lose 1.25 m of altitude in the shortest segment, it is where dense shaping and dynamics mismatch bite hardest. A pass at waypoint i is detected in the gate’s local frame—the agent’s x crossing from positive to negative while (y, z) lies inside an opening of half-side s .

Evaluation protocols. *Self-evaluation* (simulation phase): batch rollouts over thousands of parallel environments, each drawing independent perturbations of thrust-to-weight, drag and rate-loop gains. *External evaluation* (simulation phase): an independently specified protocol with fixed perturbation bounds and a reference time of 16.06 s applied to a submitted policy; we report our own measurements against its bounds rather than its outputs. *Batch evaluation* (sim-to-real phase): reset-diverse rollouts under the re-identified dynamics nominal, our primary robustness screen. *Physical evaluation*: timed three-lap runs on the real vehicle, with lap timing recovered post hoc from flight logs under a strict gate-ordering rule. **These protocols are not mutually comparable, and we do not compare them.** Simulation-phase and sim-to-real-phase times differ by roughly 2.7 s not because the policy got worse but because the phases run under different dynamics nominals (3.15 versus the value identified from flight logs, Section 5.5). A policy with less thrust authority is slower;

Table 1: PPO and network configuration by phase.

	Simulation	Sim-to-real
Actor hidden dims	[256, 256]	[512, 512, 256, 128]
Critic hidden dims	[512, 256, 128, 128]	[512, 512, 256, 256, 128, 128]
Steps per env	24	64
Mini-batches	4	8
Learning rate	5×10^{-4}	1×10^{-4}
Entropy coefficient	0.005	0.01
Empirical obs. norm.	enabled	disabled

that is physics, not regression.

3 Learning Algorithm and Infrastructure

We use PPO [11] with the clipped surrogate objective, an entropy bonus, and GAE [10] ($\gamma = 0.99$, $\lambda = 0.95$) across 8192–16384 parallel environments in the style of massively parallel on-policy learning [9]. Early prototypes collapsed around 2500 iterations, which a KL-adaptive learning rate ($\bar{d}_{KL} = 0.01$) removed; it did not remove the reward-driven collapses of Section 4.1, which were a shaping problem rather than a step-size problem.

Table 1 lists the configuration by phase. Two entries are consequences of decisions made later. The rollout length increase from 24 to 64 steps is forced by the reward becoming sparse: at 50 Hz a 24-step rollout spans 0.48 s, which frequently contains no gate-pass event, so most advantage estimates carry no task signal. And **empirical normalization** is disabled in transfer not because it hurts but because the running observation normalizer is a *learned component of the policy* the deployment controller never receives; hand-chosen fixed constants failed to recover the same performance, which we read as evidence the running statistics were doing real work—work we then had to forgo. The deeper critic follows Pasumarti et al. [7]—value estimation under sparse racing objectives benefits from more capacity than the policy needs—while actor width is fixed by the deployment controller.

4 Simulation Phase

Observation space. The policy receives 31 dimensions, spatial quantities in the body frame: body linear and angular velocity (6); gravity projected into the body frame (3); current, next and next-next gate positions (9); entry normals for the current and next gates (6); normalized waypoint index (1); sin and cos of yaw error (2); previous action (4). The gravity vector replaces the unit quaternion, preserving all tilt information the task needs without the double-cover sign ambiguity. The waypoint index is a deliberate concession to self-intersection: since position and velocity do not identify the active waypoint, the index restores the Markov property the geometry destroys, and without it the policy cannot in principle distinguish waypoint 3 from waypoint 6.

Table 2: Reward terms by phase. Progress, velocity blending, orientation penalty and the virtual-target machinery are all removed in transfer; the gate bonus is unchanged and does all the work.

Term	Sim	Sim-to-real
Gate pass (indicator)	200.0	200.0
Progress (Δdist , clamped)	50.0	off
Racing-line velocity (blended)	8.0	—
Orientation ($ \phi + \theta $ over 0.5)	-1.0	—
Lap-incomplete (per step)	—	-0.05
Rate reg. roll/pitch ($\omega_{x,y}^2$)	—	-1.0
Rate reg. yaw (ω_z^2)	—	-0.5
Action smoothness	-0.2	-0.1
Contact (per step) / death	-1.0 / -10	yes

4.1 Reward and the collapse modes it produced

Table 2 gives the converged configuration alongside its sim-to-real successor. It is the result of a search whose failures are more informative than its endpoint.

The instructive failure was a heading-alignment term (cos of yaw error) added alongside a velocity reward at scale 5.0, whose episodic contribution reached ~ 750 against the gate term’s ~ 200 . The policy became speed-dominated: it accelerated toward each gate at maximum rate and could not decelerate for the tight transition out of the central structure, gate reward collapsing from ~ 200 to ~ 40 while velocity reward rose to ~ 150 . The relevant comparison is not between reward *scales* but between *integrated episodic contributions*, and a dense term integrated over 30 s can silently dominate a sparse term worth 200 per event.

4.2 Shaping as implicit path planning

Rather than specify a geometric racing line, we shaped the velocity reward to induce one. With $\hat{d}_{\text{cur}}, \hat{d}_{\text{next}}$ the unit vectors to the current target and the following gate,

$$r_{\text{vel}} \propto \alpha \langle v, \hat{d}_{\text{cur}} \rangle + (1 - \alpha) \langle v, \hat{d}_{\text{next}} \rangle, \quad (1)$$

with $\alpha = 5/8$ on ordinary gates. Because part of the reward points at the *next* gate before the current one is reached, the optimal trajectory cuts toward the inside of each corner—smooth curving racing lines with no path parameterization, no spline and no per-corner tuning.

The blend is a genuine optimization knob with a stability boundary. Reducing α to $4/8$ —an apparently mild change—caused late-training collapse at ~ 3000 iterations, three-lap success dropping from $\sim 90\%$ to zero and mean time spiking to ~ 20 s before recovering to a degraded plateau. The next-gate component pulls the vehicle far enough off the current gate’s approach at high speed to cause misses, and once misses are common the sparse gate reward degrades faster than the dense velocity term—a self-reinforcing dynamic. We report this because α looks like a hyperparameter and behaves like a phase boundary.

Virtual targets. Because the gate-pass predicate reads the pose in the gate frame, it is independent of whatever position the reward aims at—so shaping can be steered without altering the task definition. We exploited this heavily: the central structure was designed to force a vertical loop,

a path around its *side* proved faster, and we induced that path with a two-phase virtual target placed off the gate centers entirely. The technique is powerful and does not survive contact with hardware, since it encodes a specific solution to a specific geometry under specific dynamics.

4.3 Reset distribution

Resets draw the target waypoint uniformly over all seven, so every segment trains equally, and place the vehicle behind it at a distance ramping $[1.0, 2.0] \rightarrow [1.5, 4.0]$ m over 800 iterations, with lateral, vertical and yaw noise; 40% of episodes instead start mid-segment, uniform at 20–80% of the way between consecutive gates, $\sim 20\%$ start on the ground, and half of airborne spawns receive 0.5–3.0 m/s of forward velocity.

One entry deserves emphasis: raising the mid-segment rate from 25% to 40% moved three-lap success from 86.7% to 96.3%. This was the single largest measured improvement of the simulation phase, larger than any reward change, and it was obtained by altering where episodes begin rather than what they reward. The mechanism is straightforward once stated: on-policy methods fit the value function only on the state distribution induced by ρ_0 and the current policy, so spawning behind gates trains approaches well and gate-to-gate *transitions* poorly, since transition states are visited only by trajectories that already succeeded at everything preceding them.

4.4 Randomization, paired evaluation, results

Thrust-to-weight, drag coefficients and rate-loop PID gains were resampled per environment at every reset, following standard dynamics randomization practice [13, 8]. The final model was a baseline trained 5000 iterations at 16384 environments, then fine-tuned 2000 more under widened bounds (thrust-to-weight $\pm 8\%$, drag $0.3\text{--}2.5\times$, PID $\pm 25\%$ on k_p, k_i and $\pm 40\%$ on k_d) with entropy reduced from 0.005 to 0.001.

The most instructive comparison was a *paired* evaluation: 1000 environments with identical perturbation draws presented to both models. Rank correlation between per-environment times was essentially zero (Pearson $r = -0.016$, Spearman $\rho = 0.048$). Fine-tuning under wider bounds did not translate the performance distribution; it *rotated* it. Bucketed by baseline time, the fine-tuned model was 0.87 s *slower* in the baseline’s best bucket (< 15.6 s) and 1.32, 0.99 and 1.38 s faster in the borderline, at-risk and worst buckets, converting 168 environments—78.5% of the baseline’s failures—into passes while introducing 137 new ones, a net gain of 31. A headline success-rate improvement of 2.4 percentage points therefore concealed 305 environments whose outcome changed, in both directions. Aggregate metrics on independently randomized environments cannot distinguish a uniform improvement from a reshuffle, and a reshuffle means the new model has blind spots about which the old model’s evaluation history says nothing.

Under self-evaluation across 5000 independently random-

ized environments the fine-tuned model achieved 97.3% three-lap success at 15.68 s (median 15.58, $\sigma = 0.50$, best 14.82, worst 21.26 s), against the baseline’s 94.9% and 15.78 s ($\sigma = 0.54$). Measured against the external protocol’s 16.06 s reference, 84.0% of environments finished inside it, against 82.0% for the baseline.

5 Sim-to-Real Phase

Transfer changed what was adjustable before it changed what was optimal.

5.1 The observation space becomes a contract

The deployment controller computes its own observation vector from motion-capture data, so any quantity the policy consumes must be reproducible by that code. This eliminated most of the simulation-phase observation design: engineered features—yaw error, waypoint index, gate normals—gave way to raw geometry,

$$\underbrace{v_b}_3 \mid \underbrace{\text{vec}(R_{b \rightarrow w})}_9 \mid \underbrace{C_b^{\text{cur}}}_{12} \mid \underbrace{C_b^{\text{next}}}_{12} \mid \underbrace{a_{t-1}}_4$$

for 40 dimensions, where C_b is a gate’s four corners in the body frame, computed identically on both sides as $C_w = SR_g^T + p_g$ then $C_b = (C_w - p_d)R_b$. The full rotation matrix replaces the gravity vector because, having given up the yaw-error feature, the policy needs complete attitude. Twelve numbers per gate redundantly encode a four-degree-of-freedom object, and we adopted them only because the deployment controller already computed them: *a feature that cannot be reproduced by the deployment stack is not available at any price*, and feature engineering that helps in simulation becomes a liability once the pipeline must exist twice and agree. Including the previous action follows Kaufmann et al. [3]: under actuator lag it carries information about actuator state that the kinematic observation does not.

5.2 Sparse reward redesign

Progress, velocity blending, orientation penalty and virtual targets were all removed (Table 2). The argument, following the sparse-objective results of [7] and the observation that shaping alters the optimal policy unless potential-based [6], is that dense terms specify *how* to fly and thereby encode a trajectory manifold optimal only under the training dynamics; when those dynamics shift, nothing has given the policy reason to have learned anything off that manifold. Two results generalize.

Sparse bonuses must beat integrated, not instantaneous, costs. Our first sparse configuration set the gate bonus to 10.0 and collapsed at ~ 200 iterations: the policy learned to hover. With a constant per-step penalty and a per-step contact penalty always active, and a gate taking on the order of a second to reach, the expected integrated cost of *attempting* a gate exceeded the bonus for succeeding; raising the bonus to 200.0 fixed it. A sparse bonus must exceed the integrated negative cost accumulated over

the expected time to earn it—for a 30 s episode at 50 Hz, a factor of hundreds away from the instantaneous comparison one is tempted to make.

Command regularization is a calibrated speed dial. We ablated the roll/pitch and yaw rate penalties from $(-1.0, -0.5)$ to $(-2.0, -0.05)$, since flight logs show yaw rate staying low while roll and pitch drive the aggressive transients, so the default split under-penalized the axis producing jerk. The rebalanced policy was the smoothest we trained—lowest mean tilt and body rate of any policy we flew—and reliably slower by ~ 0.6 s per lap. This is an exchange rate, not a failure: the term buys attitude margin at a measurable price in time, and it is the correct knob when margin binds.

5.3 Specification gaming on a self-intersecting task graph

Because waypoints 2, 3 and 6 share the central structure and waypoint 6 reuses waypoint 3’s plane in reverse, a naive pass predicate admits several ways to earn credit without flying the intended course. We met three. *Reverse crossing of the current target*: the vehicle crosses its target plane the wrong way, detected by the mirrored predicate and treated as termination, with a five-step cooldown after a legitimate pass since frame recomputation at the moment of passing otherwise produces false positives. *Plane-straddling at the loop gate*: the vehicle crosses waypoint 3’s plane by a few centimetres, so $x_{t-1} > 0$ and $x_t \leq 0$, then immediately reverses out, collecting the bonus without traversing the structure. The fix conditions the pass on having genuinely been in the approach zone—we track $\max_{\tau \leq t} x_\tau$ since the target last changed and require it to exceed a threshold—applied *only* at waypoint 3, because applying it globally caused visibly valid passes elsewhere to go uncounted, corrupting our own evaluation. *Backtracking through the previous gate*: after passing waypoint 2 the vehicle reverses back through its plane to re-enter the $+x$ approach region, then proceeds to waypoint 3—a cheap way to restart the approach, available precisely because waypoints 2 and 3 share a structure and a heading, and invisible to checks that monitor only the *current* target. The fix is a generic previous-gate detector on waypoint $(i - 1) \bmod 7$, which needs care at the boundaries—it must not fire before any gate is passed, nor on the step a gate is passed, when the previous-gate index has just changed and the buffered comparison is meaningless, nor outside the opening.

Splitting the reward predicate from the progress predicate. Those fixes are patches; the structural change that made them coherent was to stop using one predicate for two purposes. Earlier configurations used a single half-side s for everything, so shrinking s to encourage center-line flight also shrank the region in which the agent’s *target advanced*, producing episodes where the vehicle physically flew through a gate and the task did not acknowledge it. That is not conservatism; it is a corrupted task definition. We now maintain two predicates over the same crossing event: a **progress predicate** at the physical half-side (0.5 m), which advances the target index, increments the

lap counter and drives termination; and a **reward predicate** at a virtual inner half-side ($s/2 = 0.35$ m), which alone controls the gate bonus and insertion into the replay buffer of Section 5.4. The agent is paid for flying through the middle while the task counts what physically happened. We regard this as the most portable idea in this report: whenever a shaping term and a task-progress condition are computed from the same geometric test, tightening the shaping silently redefines the task. For the same reason we can zero dense progress at selected waypoints, since Euclidean distance to a gate center shrinks as readily from the *wrong* side as the right one.

The same exploit, two correct answers. In the simulation phase we experimented with strict termination on reverse and wrong-side passes and *deliberately removed it*: letting the agent make the mistake without instantly ending the episode produced noticeably smoother training. In transfer we added three detectors. Both were right for their regime, and the difference is what the exploit costs. Where the simulator defines correctness, an exploit costs a suboptimal trajectory while the termination suppressing it costs gradient signal and stability—frequently a bad trade. Where the simulator stands in for hardware, an exploit costs a policy that has learned a behavior the real geometry will not support, discovered at deployment. The same observation licenses opposite interventions depending on whether simulator fidelity is in question, which we think is under-appreciated in discussions of specification gaming [1] that treat exploits as unconditionally undesirable.

5.4 The reset distribution as a design object

Having found that reset placement outweighed reward tuning, we treated ρ_0 explicitly as a mixture of four components.

Gate-pass replay (30%). Following the practice of resetting to states observed during successful execution [3, 12], we maintain a per-waypoint ring buffer keyed by the *next* target gate: when the reward predicate fires, the full root state and previous action are written to the buffer for the newly acquired target, and at reset a state is drawn and perturbed by ± 0.15 m horizontally, ± 0.10 m vertically, ± 0.20 m/s linear and ± 0.50 rad/s angular velocity, ± 0.20 rad yaw. The buffer holds only states the policy actually reached, so the distribution tracks the policy as it improves; we ramp its fraction up from zero, since an unrepresentative buffer early in training supplies states the policy cannot yet handle.

Actuator-state reconstruction. A detail we consider necessary rather than optional: replaying a rigid-body state into a system with actuator dynamics does not replay the state. The rotor speeds at the recorded instant were not at rest, and initializing them to zero injects a spin-up transient into precisely the states we selected for being important. We therefore recompute the desired wrench from the replayed previous action, invert the mixer for motor-speed targets, and initialize both commanded and actual motor speeds to it, so the first step after reset is dynamically con-

tinuous with the trajectory the state came from.

Ground starts (20%). Physical evaluation begins at rest on the floor, so a fraction of episodes start there, a few centimetres up and up to 3 m behind the first gate, with a narrowed variant matching the launch pose observed in the arena. *Geometric fallback.* Remaining episodes interpolate between consecutive gates with $t \sim \text{Beta}(0.5, 0.5)$, concentrating mass near gates and thinning it mid-segment.

A cautionary ablation. The fallback spawns at rest, unlike race flight, so we added an option to give it velocity toward the target. At a cap of 1.2 m/s the policy became measurably faster but lost success rate and grew a worse time tail; at 0.6 m/s success recovered (98.3%) but the speed gain vanished entirely (mean 19.46 s against 18.34 s for the base). We had framed this as distribution matching; the result says otherwise. Rather than making training states more realistic, the higher reset velocity trained a more aggressive state distribution—buying speed by shifting the policy, not by correcting a mismatch.

5.5 Randomization as identified uncertainty

The environment randomizes the transition model—thrust-to-weight, drag, rate-loop k_p, k_i, k_d , mass, motor time constant, action latency—and the observation map: per-channel Gaussian noise ($\sigma = 0.05$ m/s on body velocity, 0.01 on rotation entries, 0.02 m on gate corners), observation latency, and systematic biases on body-frame velocity, gate-corner position and yaw. The distinction between the two groups mattered more than any individual range.

Sensitivity analysis as budget allocation. Before committing budget we ran a deterministic evaluation-only sweep over thirteen perturbation scenarios, 768 ground-start episodes each, on a single checkpoint, asking whether this policy was more likely to fail from moderate control mismatch or from systematic observation mismatch. Every transition-model perturbation left three-lap success at or above 95.6%, including 10% thrust error, 40 ms command latency and $\pm 15\%$ rate-gain error, and zero-mean observation noise at the trained magnitude was likewise harmless (96.5%). But a constant 0.20 m/s bias on observed body-frame velocity reduced success to 2.9% at a mean of 28.15 s, and a 5 cm systematic offset on gate corners cost 4.2 points and 2.1 s. Takeoff and first-gate success were 100% in all thirteen scenarios; the separating signal is entirely in later trajectory execution.

The interpretation we favor is that a closed loop rejects transition-model error as a matter of course—that is what feedback is for—whereas a systematic observation bias is an unmodeled, unobservable state offset the policy cannot detect and has no mechanism to correct. Randomizing dynamics teaches robustness to a disturbance the controller was already going to reject; randomizing observation *bias* teaches robustness to something it cannot. This is a simside result for one checkpoint, and several control scenarios scored slightly *above* nominal, which is run-to-run variation; what survives is the ordering.

Re-centering the transition model. Randomization

defends against parameters near the center of its distribution. It cannot defend against a center that is wrong. Flight logs indicated an effective mass near 64 g against the simulator’s nominal 30 g, while the deployed controller maps full policy thrust to approximately 1.174 N, so

$$\text{TWR}_{\text{real}} \approx \frac{1.174 \text{ N}}{0.064 \text{ kg} \times 9.81 \text{ m/s}^2} \approx 1.87, \quad (2)$$

against a simulator nominal of 3.15. Our widest randomization to that point was $\pm 8\%$, spanning [2.90, 3.40]. **The physical operating point was not in a low-density tail of the training distribution; it was outside its support entirely.** No width around 3.15 would have reached it, and no reward, architecture or reset change could have compensated: the policy was learning to exploit control authority the vehicle did not have.

Re-centering on 1.87 with a narrow $\pm 6\%$ band produced 98.0% success at 18.34 s ($\sigma = 0.48$) over 768 environments. Notably an *untrained-for* sanity check—the old checkpoint under the new nominal—already completed three laps at 21.16 s, which gave us confidence before committing to a retrain. The cost is specialization: the re-centered policy performs poorly at the original nominal, terminating around seven gates on representative rollouts. This is the robustness–specialization trade documented by Ferde et al. [2]—less randomization transfers faster when the nominal is correct and fails outright when it is not—and we accepted it because the deployment thrust scale is fixed and known, at the price of a policy for *this vehicle* rather than for quadrotors. Two general statements follow: randomization widens support but does not move the center, and the two are not interchangeable; and the cheapest diagnostic for a suspected support failure is not a wider sweep but a single rollout at the hypothesized nominal. Tightening the remaining ranges once the nominal was corrected (thrust-to-weight $\pm 4\%$, drag 0.8–1.2 $\times$, PID $\pm 8\% / \pm 15\%$) did not buy speed—across four candidates the best reached 98.3% at 19.46 s—which suggests the residual gap for this policy family is geometric, racing line and clearance, rather than a width effect.

6 Evaluation and Policy Selection

Three evaluation-design failures cost us more than any modeling error.

Training return is not the deployment objective. The framework saves `best_model.pt` at the argmax of mean episodic training return. Nothing about that quantity optimizes what deployment requires: three-lap success rate, low variance, a bounded worst case. In one run the saved best model was written at $\sim$ iteration 513 at mean return 5746.86; its batch evaluation was 84.7% success, 19.09 s mean, $\sigma = 1.26$ s, 28.44 s worst case—a checkpoint that wins on return by flying fast and occasionally failing badly, exactly the profile training return cannot distinguish from a good one. We replaced the rule with an explicit procedure: assemble a candidate pool spanning the saved best model, the highest-return checkpoints and late-training anchors, evaluate each under the deployment-matched protocol, and

Table 3: Backtest of evaluation profiles against known physical outcomes. Real ranking: base > speed-v4 > speed-v2.

Policy	Calib. mean (s)	Latency stress SR	Reset-diverse SR / mean (s)
base	18.24	100.0%	98.0% / 18.34
speed-v2	17.96	45.7%	96.2% / 18.68
speed-v4	18.57	15.2%	98.0% / 18.80

accept only if $\text{SR} \geq 98.5\%$, $\text{mean} \leq 18.65$ s, $\sigma \leq 0.5$ s and no pathological tail. **A faster best case is never sufficient evidence**—a rule adopted only after a candidate that won on best-case time failed in the arena.

Failure one: an evaluation distribution deployment never samples. Because our batch evaluation runs with training-mode hooks active, it inherits the *training* reset mixture: at a ground-start fraction of 0.2 a nominally 1000-environment evaluation contained roughly 210 ground starts, the other 790 beginning mid-course, at altitude, from the buffered state of a policy that had already succeeded—while every physical run begins on the floor at rest. The fix is a deployment-matched profile forcing that fraction to unity.

Failure two: an evaluation calibrated past the point of usefulness. Having identified real parameters, the natural next step was a profile that used them: identified thrust-to-weight, a fixed three-step action delay matching the measured 60–70 ms command lag, ground starts only, the narrow launch pose, and observation noise reduced from the training 0.05 m/s to the measured stationary 0.005 m/s. Since a physical session is one vehicle on one day, this seemed strictly more faithful. Backtested against three policies whose real-world ranking we knew (Table 3), it ranked *first* the policy that in the arena completed a single correctly ordered lap with negative clearance at a gate, and gave all three 100% success with $\sigma < 0.1$ s; a latency-stress variant exposed fragility but over-penalized a policy that had completed four ordered laps. The older, less faithful, reset-diverse screen correctly kept the base policy first.

The lesson is not that calibration is wrong but that an evaluation serves two purposes—estimating performance and *discriminating between candidates*—and calibrating away the diversity destroys the second. The perturbations a faithful single-vehicle profile removes are exactly the ones under which a fragile policy separates from a robust one. We now require agreement between the reset-diverse screen, targeted stress checks and physical logs before promoting a policy. The honest limit: that screen did not predict the speed-v2 failure either—it flagged lower success and much larger variance, enough to rank correctly, but the failure mode was gate clearance, which no fixed-parameter profile can surface.

7 Results

Simulation under deployment-matched conditions. Under the reset-diverse screen the deployed policy family reached 98.0–98.2% three-lap success at 18.32–18.34 s ($\sigma = 0.41$ –0.48 s). Under the deployment-matched

Table 4: Reward variants targeting lap time, under the reset-diverse screen (768 envs for base, 1000 otherwise). No variant improved mean time at matched success rate.

Candidate	SR (%)	Mean (s)	σ (s)	Best (s)
base	98.0	18.34	0.48	17.50
speed-v1	86.7	20.39	2.12	17.56
speed-v2	96.2	18.68	1.21	17.44
speed-v3	98.8	19.05	0.42	18.16
speed-v4	98.0	18.80	0.43	18.04
speed-v5	94.2	21.73	2.55	17.90

ground-only protocol (1000 ground starts) the selected checkpoint reached 100% at 18.10s with $\sigma = 0.08$ s and worst case 18.36s; two rejected alternatives also reached 100% but at 18.26s and 18.59s. The variance collapse between protocols is itself a diagnostic: nearly all reset-diverse variance comes from start-state diversity rather than from dynamics.

Speed candidates. Table 4 summarizes five attempts to recover speed by reward modification under fixed dynamics and reset settings: broad time and control pressure (v1), a post-loop velocity term on gates [4, 5, 6, 0] (v2), the same narrowed to [5, 6] (v3), a velocity term without the next-gate blend (v4), and mild time pressure alone (v5). None improved mean time at matched success rate. The candidate with the fastest single rollout (17.44s) had the second-worst variance and failed in the arena—the empirical basis for the selection rule above—and reintroducing a velocity term narrowly and weakly (v3, v4) produced stable but slower policies, suggesting the dense-to-sparse decision is not finely tunable here.

Physical evaluation. The deployed policy completed the timed three-lap evaluation in **19.004s** and continued to five correctly ordered laps at a mean lap time of 6.236s, with timing recovered from flight logs under a strict gate-ordering rule that rejects laps containing skipped or misordered gates. Against the deployment-matched prediction of 18.10s the residual gap is ~ 0.9 s, or 5%. We do not have an explanation, and want to be precise about why: gate clearance margins, gate geometry calibration and rate-loop overshoot beyond our first-order motor model are all consistent with the sensitivity analysis, a fixed-parameter evaluation profile cannot discriminate among them, and we lacked the open-loop excitation data to identify the rate loop. The gap is open. The two alternative policies flown in the same session ranked in the order predicted by the reset-diverse screen and against the order predicted by the calibrated profile, which is the evidence in Table 3.

8 Discussion and Limitations

The components most responsible for outcomes were not the ones the literature foregrounds. The learning algorithm was standard and never a bottleneck; network capacity was set by an interface constraint rather than a study; and the reward mattered mostly through a few near-discontinuous thresholds—a sparse bonus that must exceed an integrated cost, a blending coefficient with a stability boundary—rather than through smooth tuning. What determined

outcomes were the three objects this report is organized around: the reward predicate’s relationship to the progress predicate, the initial-state distribution, and the evaluation harness.

On randomization our experience supports a narrower prescription than the standard one: identify the nominal first, verify it with a single rollout at the hypothesized value before committing training time, then randomize narrowly and spend the budget where a sensitivity analysis says the policy is fragile. For us that was the observation map, not the transition model—the opposite of where dynamics-randomization practice [13, 8] directs attention, and, we suspect, a consequence of motion capture, which removes the perception gap and leaves a small, systematic, easily overlooked pipeline bias in its place [3].

The main limitation is identification. Our estimates rest on closed-loop race logs, which are policy-biased and not persistently exciting: they constrain effective thrust authority and command latency adequately and say little about motor time constant, drag, or rate-loop bandwidth. The remedy is a dedicated open-loop identification flight with a phase-structured excitation schedule—hover phases for mass and observation bias, vertical chirps for the thrust curve and motor lag, per-axis rate chirps for bandwidth, straight-line segments for drag, gate flybys for corner calibration—which we designed but did not fly. Relatedly, we never measured the real observation bias the sensitivity analysis identified as dominant, so those ranges are hypotheses, and the most direct next experiment is to measure stationary velocity bias and gate-corner error, randomize over the measured uncertainty, and re-run the sweep with elimination of the 2.9% collapse as a go/no-go criterion. We deliberately did not pursue residual dynamics learning [3]: with the nominal misidentified by a factor approaching two, a residual model would have been asked to absorb a gross parametric error, harder to fit and far harder to debug than correcting the parameter. Finally, all results are for a single track, vehicle and arena, and we have no evidence about how the ordering between observation and dynamics sensitivity would change under a perception-based state estimator, where the observation map is far noisier and far less systematic than motion capture.

References

- [1] D. Amodei et al. Concrete problems in AI safety. *arXiv:1606.06565*, 2016.
- [2] R. Ferede et al. Simulation-to-reality gap in reinforcement learning for agile flight: robustness versus specialization. *arXiv:2504.21586*, 2025.
- [3] E. Kaufmann et al. Champion-level drone racing using deep reinforcement learning. *Nature*, 620:982–987, 2023.
- [4] A. Loquercio et al. Deep drone racing: from simulation to reality with domain randomization. *IEEE Trans. Robotics*, 36(1):1–14, 2020.
- [5] M. Mittal et al. Orbit: a unified simulation framework for interactive robot learning environments. *IEEE RA-L*, 8(6):3740–3747, 2023.
- [6] A. Y. Ng, D. Harada, and S. Russell. Policy invariance under reward transformations: theory and application to reward shaping. In *ICML*, 1999.

- [7] P. Pasumarti et al. Sparse-reward reinforcement learning for autonomous drone racing. *arXiv:2512.11781*, 2025.
- [8] X. B. Peng et al. Sim-to-real transfer of robotic control with dynamics randomization. In *ICRA*, 2018.
- [9] N. Rudin et al. Learning to walk in minutes using massively parallel deep reinforcement learning. In *CoRL*, 2022.
- [10] J. Schulman et al. High-dimensional continuous control using generalized advantage estimation. In *ICLR*, 2016.
- [11] J. Schulman et al. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
- [12] Y. Song et al. Reaching the limit in autonomous racing: optimal control versus reinforcement learning. *Science Robotics*, 8(82), 2023.
- [13] J. Tobin et al. Domain randomization for transferring deep neural networks from simulation to the real world. In *IROS*, 2017.